

TypeScript Generics

Learning Objectives

- Understand what generics are and why they are important
 - Create generic functions for reusable code
 - Build generic classes and interfaces
 - Apply constraints to generic types
 - Work with multiple type parameters
 - Recognize practical use cases for generics
-

What are Generics?

Generics allow you to write code that works with multiple types while maintaining type safety. Instead of writing the same function or class for different types, you write it once with a placeholder for the type.

Real-world analogy: Think of a box that can hold any item. Instead of creating a "box for books," "box for shoes," and "box for toys," you create one generic box that can hold any type of item while still knowing what's inside.

The Problem Without Generics

```
// Without generics - not type-safe
function getFirstElement(arr: any[]): any {
  return arr[0];
}

const numbers = [1, 2, 3];
const result = getFirstElement(numbers);
// result is 'any' - we lost type information!
console.log(result.toUpperCase()); // Runtime error!
```

Output:

```
TypeError: result.toUpperCase is not a function
```

The Solution With Generics

```
// With generics - type-safe
function getFirstElement<T>(arr: T[]): T {
  return arr[0];
}

const numbers = [1, 2, 3];
const result = getFirstElement(numbers); // result is number
console.log(result);

const words = ["hello", "world"];
const word = getFirstElement(words); // word is string
console.log(word.toUpperCase());
```

Output:

1

HELLO

Generic Types: <T>

The `<T>` syntax declares a type parameter. `T` is a convention meaning "Type," but you can use any name. Common names include `T`, `U`, `K` (for key), `V` (for value).

```
// T is a placeholder for any type
function identity<T>(value: T): T {
  return value;
}

// TypeScript infers the type
const num = identity(42);      // T is number
const text = identity("hello"); // T is string
const bool = identity(true);   // T is boolean

console.log(num);
console.log(text);
console.log(bool);
```

Output:

```
42
hello
true
```

Generic Functions

Generic functions can work with any type while preserving type information.

Basic Generic Function

```
function wrapInArray<T>(value: T): T[] {
  return [value];
}

const numArray = wrapInArray(5);
const strArray = wrapInArray("hello");
const objArray = wrapInArray({ id: 1, name: "Alice" });

console.log(numArray);
console.log(strArray);
console.log(objArray);
```

Output:

```
[ 5 ]
[ 'hello' ]
[ { id: 1, name: 'Alice' } ]
```

Generic Function with Multiple Operations

```
function getLastElement<T>(arr: T[]): T | undefined {
  return arr[arr.length - 1];
}

const numbers = [10, 20, 30];
const lastNum = getLastElement(numbers);
console.log(lastNum);

const empty: string[] = [];
const lastEmpty = getLastElement(empty);
console.log(lastEmpty);
```

Output:

```
30
```

```
undefined
```

Explicitly Specifying Type Parameters

```
function createPair<T, U>(first: T, second: U): [T, U] {  
    return [first, second];  
}  
  
// Let TypeScript infer  
const pair1 = createPair("Alice", 25);  
console.log(pair1);  
  
// Explicitly specify types  
const pair2 = createPair<string, boolean>("Active", true);  
console.log(pair2);
```

Output:

```
[ 'Alice', 25 ]
```

```
[ 'Active', true ]
```

Generic Classes

Generic classes allow you to create reusable components that work with different types.

Basic Generic Class

```
class Box<T> {  
    private content: T;  
  
    constructor(value: T) {  
        this.content = value;  
    }  
  
    getContent(): T {  
        return this.content;  
    }  
  
    setContent(value: T): void {  
        this.content = value;  
    }  
}
```

```
}

const numberBox = new Box<number>(100);
console.log(numberBox.getContent());

const stringBox = new Box<string>("Hello");
console.log(stringBox.getContent());
stringBox.setContent("World");
console.log(stringBox.getContent());
```

Output:

```
100
Hello
World
```

Generic Class with Methods

```
class Stack<T> {
  private items: T[] = [];

  push(item: T): void {
    this.items.push(item);
  }

  pop(): T | undefined {
    return this.items.pop();
  }

  peek(): T | undefined {
    return this.items[this.items.length - 1];
  }

  isEmpty(): boolean {
    return this.items.length === 0;
  }
}

const numberStack = new Stack<number>();
numberStack.push(10);
numberStack.push(20);
console.log(numberStack.peek());
console.log(numberStack.pop());
console.log(numberStack.isEmpty());
```

Output:

```
20
20
false
```

Generic Interfaces

Generic interfaces define contracts that can work with multiple types.

Basic Generic Interface

```
interface Container<T> {
    value: T;
    getValue(): T;
    setValue(val: T): void;
}

class NumberContainer implements Container<number> {
    constructor(public value: number) {}

    getValue(): number {
        return this.value;
    }

    setValue(val: number): void {
        this.value = val;
    }
}

const container = new NumberContainer(42);
console.log(container.getValue());
container.setValue(100);
console.log(container.getValue());
```

Output:

```
42
100
```

Generic Interface for API Response

```
interface ApiResponse<T> {
    data: T;
    status: number;
    message: string;
```

```
}

interface User {
  id: number;
  name: string;
}

const userResponse: ApiResponse<User> = {
  data: { id: 1, name: "Alice" },
  status: 200,
  message: "Success"
};

console.log(userResponse.data.name);
console.log(userResponse.status);
```

Output:

Alice

200

Generic Constraints

Constraints limit which types can be used with generics. Use the `extends` keyword to specify constraints.

Constraining to Specific Properties

```
interface HasLength {
  length: number;
}

function logLength<T extends HasLength>(item: T): void {
  console.log(`Length: ${item.length}`);
}

logLength("Hello");           // string has length
logLength([1, 2, 3]);         // array has length
logLength({ length: 10 });    // object with length

// logLength(123); // Error: number doesn't have length
```

Output:

```
Length: 5
Length: 3
Length: 10
```

Constraining to Object Types

```
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

const person = {
  name: "Alice",
  age: 30,
  city: "New York"
};

console.log(getProperty(person, "name"));
console.log(getProperty(person, "age"));

// getProperty(person, "salary"); // Error: 'salary' doesn't exist
```

Output:

```
Alice
30
```

Constraining to Class Types

```
class Animal {
  constructor(public name: string) {}
}

class Dog extends Animal {
  bark(): void {
    console.log("Woof!");
  }
}

function createInstance<T extends Animal>(
  AnimalClass: new (name: string) => T,
  name: string
): T {
  return new AnimalClass(name);
}
```

```
const dog = createInstance(Dog, "Buddy");
console.log(dog.name);
dog.bark();
```

Output:

Buddy

Woof!

Multiple Generic Parameters

You can use multiple type parameters in functions, classes, and interfaces.

Function with Two Type Parameters

```
function merge<T, U>(obj1: T, obj2: U): T & U {
  return { ...obj1, ...obj2 };
}

const result = merge(
  { name: "Alice" },
  { age: 30, city: "NYC" }
);

console.log(result.name);
console.log(result.age);
console.log(result.city);
```

Output:

Alice

30

NYC

Generic Class with Multiple Parameters

```
class Pair<K, V> {
  constructor(public key: K, public value: V) {}

  display(): void {
    console.log(`${this.key}: ${this.value}`);
  }
}
```

```
const pair1 = new Pair<string, number>("age", 25);
pair1.display();

const pair2 = new Pair<number, string>(1, "first");
pair2.display();
```

Output:

```
age: 25
```

```
1: first
```

Generic Map Structure

```
class Dictionary<K, V> {
  private items: Map<K, V> = new Map();

  set(key: K, value: V): void {
    this.items.set(key, value);
  }

  get(key: K): V | undefined {
    return this.items.get(key);
  }

  has(key: K): boolean {
    return this.items.has(key);
  }
}

const dict = new Dictionary<string, number>();
dict.set("apple", 5);
dict.set("banana", 3);

console.log(dict.get("apple"));
console.log(dict.has("orange"));
```

Output:

```
5
```

```
false
```

Practical Use Cases for Generics

1. Data Fetching Utility

```
async function fetchData<T>(url: string): Promise<T> {
  const response = await fetch(url);
  const data: T = await response.json();
  return data;
}

// Usage example (simulated)
interface Product {
  id: number;
  name: string;
  price: number;
}

// const products = await fetchData<Product[]>('/api/products');
// TypeScript knows products is Product[]
```

2. Generic Array Operations

```
function filterArray<T>(
  arr: T[],
  predicate: (item: T) => boolean
): T[] {
  return arr.filter(predicate);
}

const numbers = [1, 2, 3, 4, 5, 6];
const evenNumbers = filterArray(numbers, n => n % 2 === 0);
console.log(evenNumbers);

const words = ["hi", "hello", "hey", "goodbye"];
const shortWords = filterArray(words, w => w.length <= 3);
console.log(shortWords);
```

Output:

```
[ 2, 4, 6 ]
```

```
[ 'hi', 'hey' ]
```

3. Generic Repository Pattern

```
interface Entity {
  id: number;
}

class Repository<T extends Entity> {
  private items: T[] = [];

  add(item: T): void {
    this.items.push(item);
  }

  findById(id: number): T | undefined {
    return this.items.find(item => item.id === id);
  }

  getAll(): T[] {
    return this.items;
  }
}

interface User extends Entity {
  name: string;
}

const userRepo = new Repository<User>();
userRepo.add({ id: 1, name: "Alice" });
userRepo.add({ id: 2, name: "Bob" });

console.log(userRepo.findById(1));
console.log(userRepo.getAll().length);
```

Output:

```
{ id: 1, name: 'Alice' }
```

```
2
```

4. Generic Event Emitter

```
class EventEmitter<T> {
  private listeners: Array<(data: T) => void> = [];

  subscribe(listener: (data: T) => void): void {
    this.listeners.push(listener);
  }

  emit(data: T): void {
    this.listeners.forEach(listener => listener(data));
  }
}
```

```

}

interface Message {
  text: string;
  sender: string;
}

const messageEmitter = new EventEmitter<Message>();

messageEmitter.subscribe((msg) => {
  console.log(`${msg.sender}: ${msg.text}`);
});

messageEmitter.emit({ text: "Hello!", sender: "Alice" });
messageEmitter.emit({ text: "Hi there!", sender: "Bob" });

```

Output:

```
Alice: Hello!
```

```
Bob: Hi there!
```

Practice Exercise

Create a generic `Queue` class with methods `enqueue`, `dequeue`, `peek`, and `size`. Test it with both numbers and strings.

```

class Queue<T> {
  private items: T[] = [];

  enqueue(item: T): void {
    this.items.push(item);
  }

  dequeue(): T | undefined {
    return this.items.shift();
  }

  peek(): T | undefined {
    return this.items[0];
  }

  size(): number {
    return this.items.length;
  }
}

// Test with numbers
const numberQueue = new Queue<number>();

```

```
numberQueue.enqueue(10);
numberQueue.enqueue(20);
numberQueue.enqueue(30);
console.log("Peek:", numberQueue.peek());
console.log("Dequeue:", numberQueue.dequeue());
console.log("Size:", numberQueue.size());

// Test with strings
const stringQueue = new Queue<string>();
stringQueue.enqueue("first");
stringQueue.enqueue("second");
console.log("Peek:", stringQueue.peek());
console.log("Size:", stringQueue.size());
```

Output:

```
Peek: 10
Dequeue: 10
Size: 2
Peek: first
Size: 2
```

Summary

In this module, you learned:

- What generics are and why they provide type safety with flexibility
- How to create generic functions with type parameters
- Building generic classes for reusable components
- Defining generic interfaces for contracts
- Applying constraints to limit generic types
- Working with multiple type parameters
- Practical patterns: repositories, event emitters, data fetching

Generics are essential for writing reusable, type-safe code in TypeScript. They allow you to create flexible components that work with any type while maintaining full type checking. Practice these patterns to build robust applications.